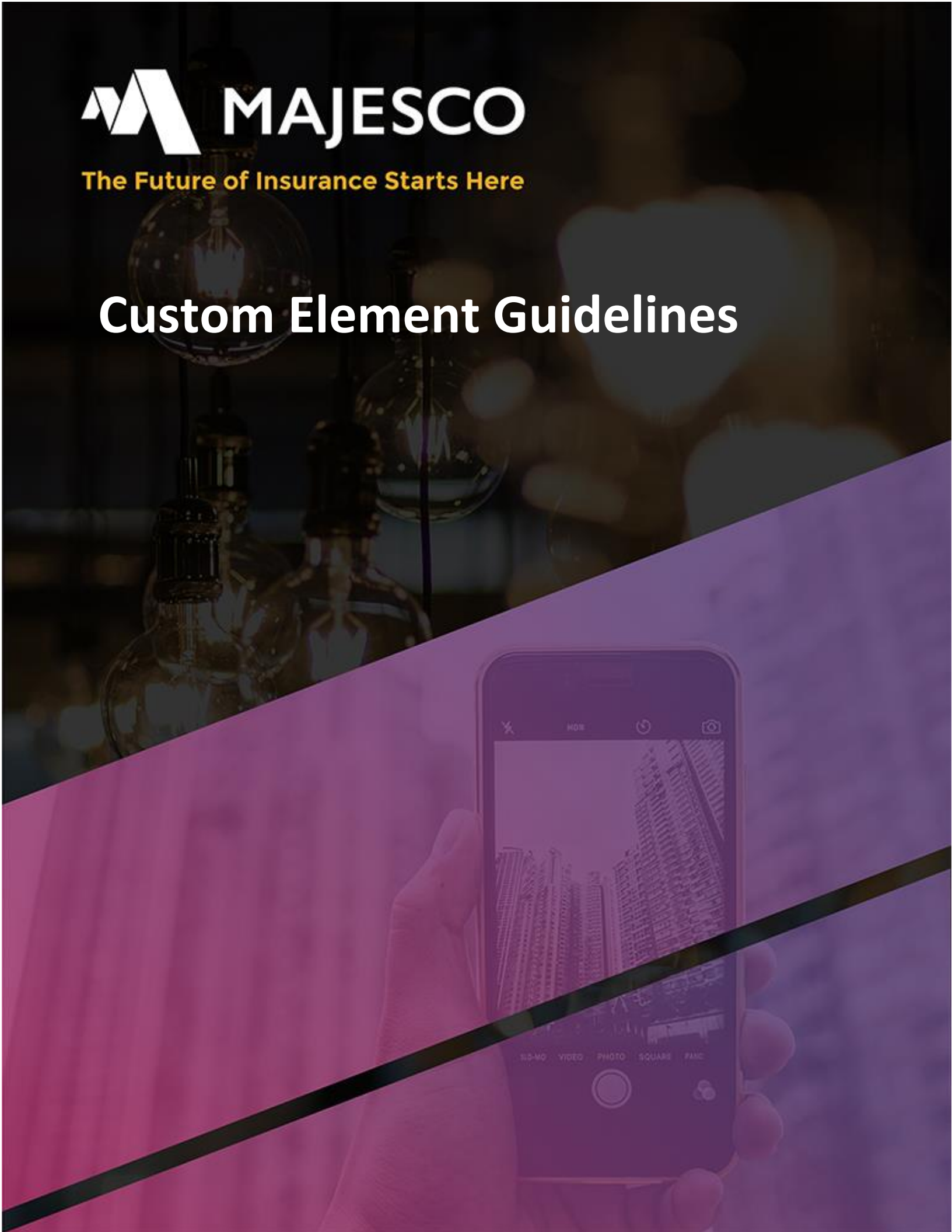
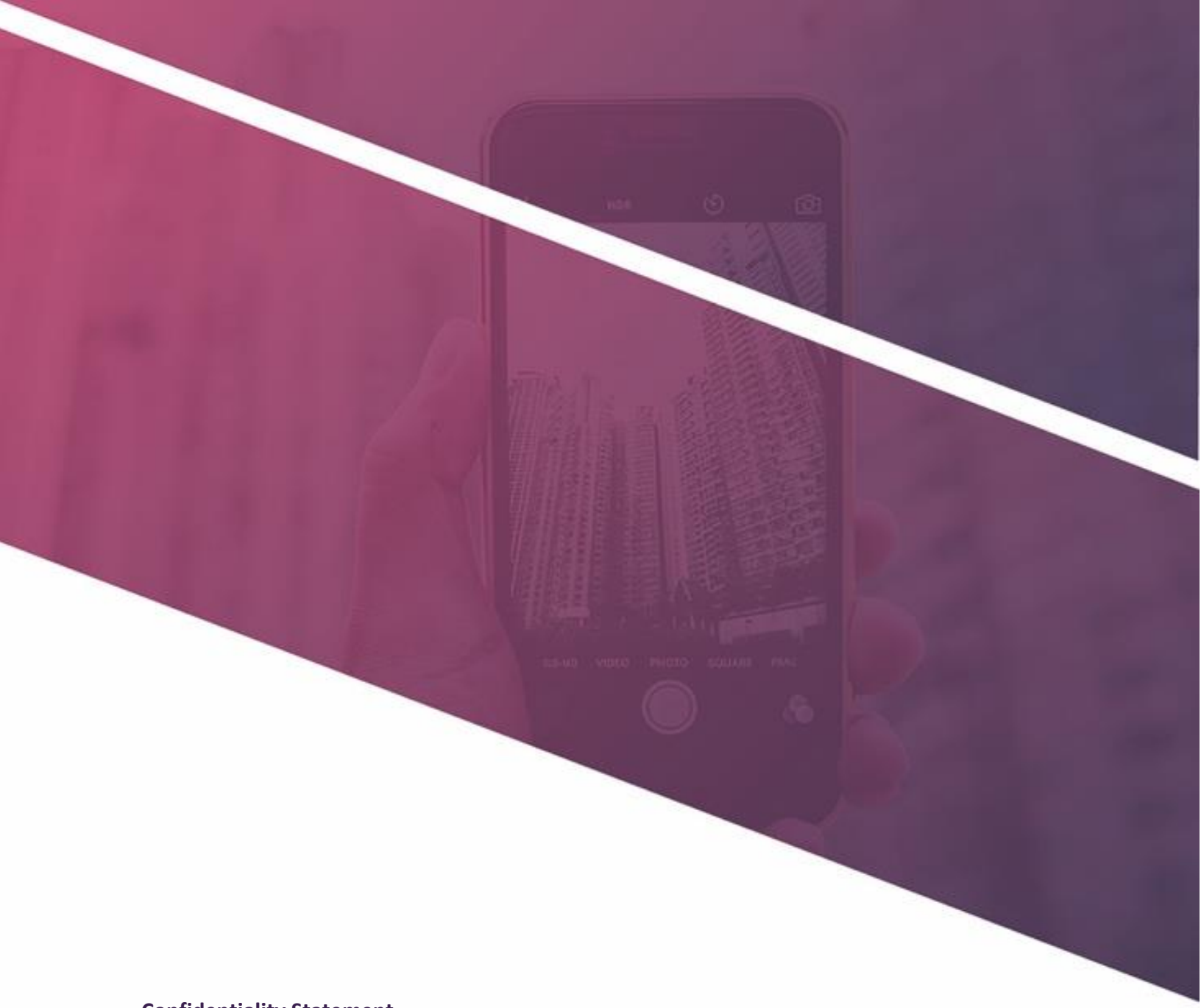
MAJESCO

The Future of Insurance Starts Here

Custom Element Guidelines





Confidentiality Statement

This document contains proprietary and confidential information and data of Majesco Ltd. and its affiliates. This document is provided, on the express condition that such information and data will not be used, disclosed, or reproduced in any form, in whole or in part, for any purpose (other than solely for evaluation purposes by authorized representatives with a need to know), without the express written approval of Majesco. The right to use, disclose, and reproduce such information and data shall be governed by the service agreement between the client and Majesco.

Warnings and Disclaimer

The information provided is on an as is basis. Every effort has been made to make this book as complete and as accurate as possible, but no warranty or fitness is implied. The authors and the publisher shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained in this book. Screenshots and data used within are for illustration purpose and may vary from the actual application.

© 2021 Majesco. All Rights Reserved.

MAJESCO

Table of Contents

1 Accessibility & Compliance 5

2 Browser Compatibility 6

3 Internationalization/Localization: 7

4 Standardized Naming Convention for Properties and Values: 8

5 Responsive 9

6 Coding Standard 10

7 Standard Properties Naming Convention and Support 11

7.1 Required Indicator and Required Indicator Style 11

7.2 Placeholder 11

7.3 Updatable 11

7.4 Label Positioning and Label Alignment 11

7.5 Special Characters 11

8 Theming/Styling 12

9 External Code Editor 13

10 Third-Party Library usage (JavaScript or CSS) 14

10.1 OOB available libraries 14

11 Cleanup 15

12 No Multiple Process calls 16

13 Appendix 1 – Custom Widget 17

13.1 Widget Input Data 17

13.2 Widget Properties 17

© 2021 Majesco. All Rights Reserved.

13.3 Unique HTML Element Id 17
13.4 APIs 17
13.4.1 Event Notification to Platform 17
13.4.2 HTTP Services 18
13.4.3 Data Formatting Service 19
13.5 JavaScript Coding Guidelines 22
13.5.1 Dont's 22

© 2021 Majesco. All Rights Reserved.

1 Accessibility & Compliance

- WCAG Compliance. Execute WAVE tool locally before any release.
- Use HTML5 Semantic elements/tags.
- Make sure you always give preference to HTML native controls rather than creating custom control using div and span.
- 'Alt text' must be provided with an image whenever it is used inside your component.
- If a component (control) is not created using any native HTML tag than kindly make use of 'ARIA' for accessibility. For example, Checkbox with toggle is created with normal div so marked it as "checkbox control" using ARIA. It makes easier for a Screen reader to understand controls functionality.
- If your custom control requires a focus means control is not created using native HTML tags and need a focus than make use of "tabindex=0" attribute. To make focus visible kindly provide class "control-has-focus" to your custom control.
- Any component works based on the session or time limit than user must be provided with any of the following provision:
  (Make sure the time extension should not be provided when it is meant to be as a feature. For example, for an auction, for time base competition, etc..)
  - The user is allowed to turn off the time limit
  - The user is allowed to adjust the time limit.
  - The user is warned before time expires and given at least 20 seconds to extend the time limit with a simple action.

© 2021 Majesco. All Rights Reserved.

2 Browser Compatibility

- Chrome
- Edge
- Firefox
- Safari
- IE11

Refer to Official Browser Supported.pdf

© 2021 Majesco. All Rights Reserved.

3 Internationalization/Localization:

- Use widgetInputData.i18nPipe.transform
  - To convert any static text you may use in the custom element.
  - To convert values for properties, subjected to internationalization/localization.
  - For dropdown options and option-group.

© 2021 Majesco. All Rights Reserved.

4 Standardized Naming Convention for Properties and Values:

Example: All custom widget should expose property named PLACEHOLDER for placeholder.

For REQ_INDICATOR_STYLE values use standard List-of-values.

All "Label" related properties should start with "LABEL_".

© 2021 Majesco. All Rights Reserved.

5 Responsive

- Avoid fix pixels for structuring unless specified.
- Follow responsive guidelines
- Do not use padding or margin for horizontal or vertical alignment.

© 2021 Majesco. All Rights Reserved.

6 Coding Standard

- Use "Const" Constants in lieu of String literals
  - For Properties, define constant and use it to access property value from widgetPropeties.
  - Use format "PROP_<PropertyName>_V". E.g. PROP_UPDATABLE_V = 'UPDATABLE'
  - Use it to access property values in JS. E.g. widgetProperties[PROP_UPDATABLE_V]
  - String literal e.g. const SUCCESS_V = 'SUCCESS';
- Use Standard Javascript naming convention for variables and functions.
  - E.g. For variables use: let myVariable
  - E.g. For function use : function myFunction
  - For Common library, add "Fn" suffix to the function name. This helps to visually distinguish function from variables. E.g. function myFunctionFn

Usually common constants needs to be defined in the common library. Follow the "Const" declaration coding guideline.

- Use spaces around operators E.g. x = y + z;
- Always end statement with semicolon.
- Do not use "var". Use "let" or "const" for defining variables.
- Do not use ES2015/ES6 standard (for IE11 compatibility)
  - Do not use "Template Literals"
  - Do not use "=>" (shorthand notation for function declaration)
  - Do not use 'Const' in for loop statement. E.g. (for const x in y)
  - Check browser compatibility on https://developer.mozilla.org/ or https://caniuse.com/
- Whenever using callback functions, use bind(this).
- Do not use document.ready for any jQuery initialization.
- Do not copy paste code blindly from web. Understand the code and then use.

© 2021 Majesco. All Rights Reserved.

7 Standard Properties Naming Convention and Support

7.1 Required Indicator and Required Indicator Style

- Property Name: REQ_INDICATOR and REQ_INDICATOR_STYLE (Not added automatically)
- Editor Type: LIST
- List of Options: REQ_INDICATOR and REQ_INDICATOR_STYLE
- Need to implement behavior in given widget.

7.2 Placeholder

- Property Name: PLACEHOLDER (Not added automatically)
- Editor Type: TEXT
- Need to implement behavior in given widget.

7.3 Updatable

- Property Name: UPDATABLE (For widget, this is added automatically)
- Need to implement behavior in given widget; which for input control involves setting "disabled=true".

7.4 Label Positioning and Label Alignment

- Property Name: LABEL_POSITION and LABEL_ALIGNMENT (Not added automatically)
- Editor Type: LIST
- List of Options: LABEL_POSITION and LABEL_ALIGNMENT
- Need to implement behavior in given widget.

7.5 Special Characters

- Property Name: CHARACTERS_NOT_PERMITTED(Not added automatically)
- Editor Type: TEXT (accept comma separated characters. Assuming comma is allowed)
- Need to implement behavior in given widget.

© 2021 Majesco. All Rights Reserved.

8 Theming/Styling

- For existing widgets refer to Theme Variables
- Use/Create CSS variables.
- Create Application wide CSS with CSS-rules and CSS custom properties.
- Use data-* attributes for styling based on property values wherever possible.
- Follow CSS Naming convention (Block-Element_Modifier)

© 2021 Majesco. All Rights Reserved.

9 External Code Editor

If need arise, Use Visual Studio Code (VS Code) for editing widget source outside Digital1st.

© 2021 Majesco. All Rights Reserved.

10 Third-Party Library usage (JavaScript or CSS)

10.1 OOB available libraries

- Jquery
- Bootstrap 3
- Lodash
- Choices.js
- Toastr.js

Before adding/using new library, please get necessary approval/review.

© 2021 Majesco. All Rights Reserved.

11 Cleanup

Implement following hook/function to clean up memory, DOM references, Subscriptions etc.

```
function dgOnDestroy() {
}
```

© 2021 Majesco. All Rights Reserved.

12 No Multiple Process calls

If widget is data bound widget, meaning object mapping is honored and value can be changed; and also an event is configured to one of the trigger (i.e. Button Click or Value Change or Enter key press) then please make sure it does cause multiple "process" calls.

© 2021 Majesco. All Rights Reserved.

13 Appendix 1 – Custom Widget

13.1 Widget Input Data

Every widget is provided an object called "widgetInputData", which consists of following:

- Widget Properties
- Unique HTML Element Id
- Various Services

This can be accessed in the custom widget Javascript as "this.widgetInputData.<something>".

13.2 Widget Properties

Access property exposed by widget in javascript as "*this.widgetInputData.widgetProperties.LABEL*" (provided LABEL is the property exposed by widget).

In HTML, properties can be accessed as "*widgetProperties.LABEL*". i.e. without "this.widgetInputData"

13.3 Unique HTML Element Id

Use "*Unique HTML element Id*" to set Ids for HTML elements created in custom element. This is required to maintain uniqueness of Ids.

E.g.

```
<button id="{{uniqueHTMLElementId}}_dynamic_template" class="someClass">{{wid
getProperties.LABEL}}</button>
```

13.4 APIs

13.4.1 Event Notification to Platform

Click, Value Change and Keyup events can be notified to Platform using following APIs

#	API Name	Usage
1	dispatchClickEvent	this.widgetInputData.eventAPIService.dispatchClickEvent(this.widgetInputData.mountPointElement, {dataValue: <actual value>});

© 2021 Majesco. All Rights Reserved.

#	API Name	Usage
		- Where <actual value> is the value one wants to set in the mapped Object Field.
2	dispatchValueChangeEvent	this.widgetInputData.eventAPIService.dispatchValueChangeEvent(this.widgetInputData.mountPointElement, {dataValue: <actual value>}); - Where <actual value> is the value one wants to set in the mapped Object Field.
3	dispatchActionKeyEvent	this.widgetInputData.eventAPIService.dispatchActionKeyEvent(this.widgetInputData.mountPointElement, {dataValue: <actual value>}); - Where <actual value> is the value one wants to set in the mapped Object Field.

13.4.2 HTTP Services

Use Http service to make GET and POST http calls.

#	API Name	Usage

1	get	this.widgetInputData.httpAPIService.get(httpServiceOption);
		
		Input: httpServiceOption should be an object as follows:
		{
		url: <api url to be invoked>,
		headers: {<header name1>: <header value1>, <header name2>: <header value2>},
		body: <body to be passes to api>
		}
		
		Returns Promise of response from invoked api.

2	post	this.widgetInputData.httpAPIService.post(httpServiceOption);
		
		httpServiceOption can be an object as follows:
		{
		url: <api url to be invoked>,
		headers: {<header name1>: <header value1>, <header name2>: <header value2>},
		body: <body to be passes to api>
		}
		
		Returns Promise of response from invoked api.

3	getEcoExchangeParameters	this.widgetInputData.httpAPIService.getEcoExchangeParameters(httpServiceOption);

© 2021 Majesco. All Rights Reserved.

#	API Name	Usage

		httpServiceOption can be an object as follows:
		{
		ecoExchangeAppName: : <eco exchange app name for which parameters needs to be queries>
		}
		
		Returns Promise, which when resolved, looks like as follows:
		
		{
		Parameter1: value1,
		Parameter2: value2
		}

13.4.3 Data Formatting Service

You can access using widgetInput data : this.widgetInputData.dataFormattingService;\

Usage

Example:
let DataFormatDetails = {
dataFormat: undefined,
dataType: this.widgetInputData.objectProperties.DATA_TYPE,
dataValue: <value to be formatted>,
dynamicFormat: <Dynamic Format>,
iceMode: this.widgetInputData.iceMode,
locale: this.widgetInputData.appLocale,
timeZoneId: this.widgetInputData.appTimeZoneId
};
 	
<value to be formatted> = this.widgetInputData.dataFormattingService.formatData(DataFormatDetails);
 	
Where,
<value to be formatted> is Value which you want to format.
<Dynamic Format>, pass format value as specified in below confluence link. Format that you want to apply.

Make sure format will be specified as shown in below:

Text

Numeric Format
The following is the list of formats supported in Numeric Format field for the property 'Dynamic Number Format'

© 2021 Majesco. All Rights Reserved.

Localization can be applied to formats which are numeric strings by appending 'L' to the format.

Format	Example String	Formatted String	Format with Localization
Numbers			
'0,0.0000'	10000	10,000.0000	
'0,0'	10000.23	10,000	
'+0,0'	10000.23	+10,000	
'0,0.0'	-10000	-10,000.0	
'0.000'	10000.1234	10000.123	
'00000'	100.1234	00100	'00000L'
'000000,0'	1000.1234	001,000	
'000.00'	10	010.00	'000.00L'
'0[.]00000'	10000.1234	10000.12340	
'(0,0.0000)'	-10000	(10,000.0000)	
'.00'	-0.23	-.23	'.00L'
'(.00)'	-0.23	(.23)	
'0.00000'	0.23	0.23000	'0.00000L'
'0.0[0000]'	0.23	0.23	
'0.0a'	1230974	1.2m	
'0 a'	1460	1 k	
'0a'	-104000	-104k	
'0o'	1	1st	
'0o'	100	100th	
Currency			
'$0,0.00'	00.234	$1,000.23	
'0,0[.]00 $'	1000.2	1,000.20 $	
'$ 0,0[.]00'	1001	$ 1,001	
'($0,0)'	-1000.234	($1,000)	
'$0.00'	-1000.234	-$1000.23	
'($ 0.00 a)'	1230974	$ 1.23 m	
Bytes			
'0b'	100	100B	
'0b'	1024	1KB	
'0 ib'	2048	2 KiB	

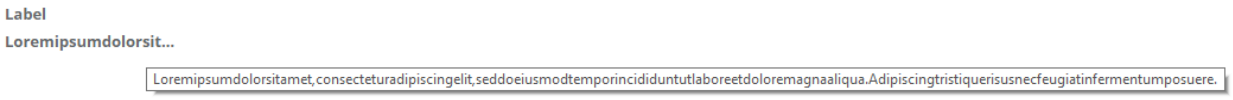
© 2021 Majesco. All Rights Reserved.

Format	Example String	Formatted String	Format with Localization
'0.0 b'	3072	3.1 KB	
'0.00b'	7884486213	7.88GB	
'0.000 ib'	3467479682787	3.154 TiB	
Percentages			
'0%'	1	100%	
'0.000%'	0.974878234	97.488%	
'0 %'	-0.43	-43 %	
'(0.000 %)'	0.43	43.000 %	
Time			
'00:00:00'	25	0:00:25	
'00:00:00'	238	0:03:58	
'00:00:00'	63846	17:44:06	
Exponential			
'0,0e+0'	1123456789	1e+9	
'0.00e+0'	12398734.202	1.24e+7	
'0.000e+0'	0.000123987	1.240e-4	

Setting Ellipsis property to text

Following properties need to be configured, in order to truncate overflowing text with ellipsis and show complete text in tooltip.

- Max Width : set the max width of the text
- Show Tooltip: set the proerty to 'Yes'



NoClientIdentifier:

When any API is called using Http Service, "clientIdentifier" is automatically sent in the request header.
However, if need is to not to send the "clientIdentifier" then one can set header explicitly in "httpServiceOption" as follows:

```
{
  url: <api url to be invoked>,
  headers: {'NoClientIdentifier': 'true'},
```

© 2021 Majesco. All Rights Reserved.

```
  body: <body to be passes to api>
}
```

13.5 JavaScript Coding Guidelines

JavaScript for custom widget needs to have following functions/methods.

- connect
  This is where some initialization logic can be written.
- paint
  This is where usually Javascript logic will be written.

E.g.

```
this.attachClickListener = attachClickListener;
function connect() {
 this.paint();
}
function paint() {
 this.attachClickListener();
}
function attachClickListener() {
 $("#"+this.widgetInputData.uniqueHTMLElementId).click(function(event) {
 this.widgetInputData.eventAPIService.dispatchClickEvent(this.widgetInputData
.mountPointElement, {dataValue: event.target.value});
 });
}
```

13.5.1 Dont's

Do not use ECMA2015 syntax for IE 11 compatibility.

© 2021 Majesco. All Rights Reserved.